

Multi-resolution semantic abstraction over the TKH hypergraph

Ilia Dzeneladze, 2026-09-21

I designed and directed this project end to end, using a coding agent (Claude Code) to implement, run, and debug the pipeline under my direction. For a few parts genuinely past what I'd expect to know starting this, mainly the hypergraph spectral affinity math and a couple of subtle correctness bugs, I worked through the reasoning with the agent directly rather than just accepting output. Specifics on tool use and prompts are in [AI_USAGE.md](#).

0. Formal statement

Objective. Given a hypergraph snapshot $H(t) = (V(t), E(t))$ restricted to "concept" node types (method, technique, task, problem, dataset, metric, component, cited_work, future_topic, claim; article and author shape structure but never become super-node members themselves), build a laminar sequence of partitions $P_0, \dots, P_K$ of $V(t)$ by agglomerative clustering on a single joint affinity graph

$$A(i, j) = \alpha * S(i, j) + (1 - \alpha) * M(i, j)$$

where S is a hypergraph-native structural affinity (each hyperedge e of arity n contributes $1/(n-1)$ to every pair of its endpoints, following Zhou, Huang and Schoelkopf's normalized hypergraph cut) and M is a semantic k -NN affinity over sentence embeddings of node surface_form text. Both are normalized to $[0,1]$ independently before combining, $\alpha = 0.3$ in the final run, chosen by ablation (section 3) rather than left at the 0.5 I started with. The dendrogram from average-linkage clustering on A is cut at three target sizes (12, 50, 200) to produce three super-node levels; the tree itself is the laminar hierarchy, cuts of one dendrogram at different heights are always nested in each other.

The structure/semantics trade-off (P3). I resolved this as a single joint objective rather than a two-stage or consensus scheme, mainly because it collapses the trade-off into one literal, sweepable parameter (α) instead of a qualitative staging decision about which signal gets to act first. I considered a two-stage alternative (spectral structural clustering fixes the coarse level, semantics only refines within it) and rejected it: it's computationally convenient, but it means the coarse level is determined by structure alone, so a case where two topically identical but citation-disconnected clusters should merge at a coarse level has no path to do so. The joint graph lets a strong semantic signal override weak structural evidence at any level, not just at the bottom. The real cost of the joint approach, as originally shipped, was that α itself was never properly ablated, it sat at 0.5 by default. I closed that specific gap afterward with a full sweep (section 3), which is how the final run ended up at $\alpha=0.3$ instead. The two-stage alternative above is still only reasoned about, not actually implemented and measured against the shipped method; that comparison remains a real gap (section 5).

Complexity. Structural affinity construction is $O(\sum_e |e|^2)$ in the worst case (bounded by roughly 66k nonzero pairs on the largest snapshot, 5428 concept nodes, 1429 hyperedges). The semantic k -NN graph is a $k=15$ nearest-neighbor search over 768-dim embeddings, dominated in practice by the sentence-embedding forward pass itself (about 107 of 110 seconds on the full 2026 snapshot) rather than the neighbor search. The sparse UPGMA merge is $O(E \log E)$ via a heap with lazy deletion, and empirically fast, about 1.6 seconds for 5427 merges on the same snapshot. Wall-clock at this corpus size is embedding-bound, not algorithm-bound.

Guaranteed by construction vs. measured empirically.

- P1 (laminar refinement): by construction. A dendrogram's cuts are nested by definition; `validate_hierarchy.py` checks this exactly (every super-node's children partition its members with zero gap or overlap) across all four snapshots, not just assumed from the algorithm's shape.
- P2 (size budget, N_0 in $[10,15]$): by construction. `cut_to_k_clusters` always returns at most the target count; $N_0 = 12$ at every snapshot.
- P3 (semantic coherence): empirical, not guaranteed. The joint-affinity weighting is a design choice with no mathematical guarantee that meaning wins out over noise. What's measured (section 4 below) is strongly positive, but that's a result, not a property of the construction.
- P4 (hyperedge fidelity): by construction, in two places. The structural affinity itself is computed directly from hyperedge incidence (never projects to a pairwise graph before that point), and the T4 collapse rule explicitly keeps a k -ary relation as one coarse hyperedge when it spans three or more super-nodes rather than exploding it into a clique. Both are implemented, not just described, and both are unit tested.

- P5 (temporal stability): empirical, and deliberately not optimized for. The matching mechanism (Jaccard overlap across independently-clustered snapshots) only measures stability after the fact, it does nothing during clustering to encourage it. Measured cross-snapshot ARI ranges 0.35 to 0.65 depending on level (section 4).
- P6 (interpretable, faithful, temporally honest labels): mixed. Temporal honesty for the labeller's INPUT is by construction, a label for year t structurally cannot see member text the corpus hadn't ingested by t , because that node was never included in the snapshot in the first place. Whether the label itself avoids overclaiming given that input is empirical and imperfect: the measured over-claim rate is 4.8-11.3% across snapshots, not zero.

1. Method and where it sits in the literature

The structural affinity follows Zhou, Huang and Schoelkopf (2006), the standard spectral formulation of hypergraph clustering via a normalized hypergraph Laplacian, generalizing pairwise spectral clustering to hyperedges without first collapsing them to pairs. I checked what a naive unweighted clique expansion would have done instead: on the 2026 snapshot, hyperedges of arity greater than 10 hold about 94% of total pairwise weight under the naive version, versus about 63% under the native $1/(n-1)$ weighting. Since roughly 80% of the hyperedges in this corpus have arity greater than 2, that's not a theoretical nicety, a handful of large tables would otherwise dominate the whole structural signal.

I considered hypergraph modularity (Kumar et al. 2020) as an alternative clustering objective, closer in spirit to Louvain-style greedy community detection than to spectral cuts. I didn't use it, mainly because a modularity-maximizing method doesn't naturally hand you a full laminar hierarchy the way a dendrogram does, you'd need to run it recursively at multiple resolutions and then separately verify the results nest properly, which is exactly the kind of "assumed, not verified" risk I was trying to avoid by using average-linkage clustering directly.

The temporal matching mechanism (birth, death, stable, grow, shrink, merge, split) follows the event vocabulary from Greene, Doyle and Cunningham (2010) on tracking community evolution in dynamic networks, applied here via Jaccard overlap between independently-clustered snapshots rather than their original setting. I chose matching over warm-starting the clustering itself (which would be closer to a regularized/temporally-smoothed method) because it keeps every snapshot fully reproducible in isolation. The real cost, stated plainly: matching only measures stability, it never pushes the clustering toward being stable, so whatever temporal coherence shows up in the results is earned by the corpus's own structure and the labels' persistence, not engineered in.

For the coarsening operation itself (T4, next section), the closest published framing is graph reduction with spectral guarantees (Loukas 2019), though that work is about pairwise graphs and doesn't address what happens to an edge that spans three or more coarse groups at once, which is the genuinely hypergraph-specific part of T4.

I stuck to citations I could verify the details of rather than reaching for very recent (2023-2026) hypergraph clustering papers I wasn't fully sure I remembered correctly. That's a real limitation of the literature grounding here, not a claim that nothing newer is relevant.

2. The T4 coarsening decision

When a hyperedge's endpoints land across super-nodes after clustering, three cases: all endpoints in one super-node means the edge is now fully internal, dropped from the coarse hypergraph and counted instead as an internal-density statistic; endpoints spanning exactly two super-nodes becomes an ordinary pairwise edge; endpoints spanning three or more super-nodes stays a genuine k -ary hyperedge between those groups. That last case is the one worth defending: the lossy alternative is exploding it into every pairwise combination between the spanned groups, which inflates apparent density by $O(k^2)$ and destroys the fact that all k groups were asserted together by one source fact, not by $k(k-1)/2$ independent facts.

This isn't just a written rule, it's what the pipeline actually applies at every level of every snapshot. On the 2026 snapshot's level-0 hierarchy (12 super-nodes), of 1429 original hyperedges, 592 became coarse edges: 128 pairwise and 464 genuine multi-way hyperedges, only 7 became fully internal and were dropped. What the rule loses: once several original hyperedges collapse onto the same super-node set, they merge into one coarse edge with a weight count and a relation-type breakdown, not a record of which specific original member participated in which specific original edge.

These exact counts shifted after the alpha ablation in section 3 (alpha=0.5 gave 532 coarse edges with 116 internal-dropped at level 0; alpha=0.3 gives 592 with only 7 dropped). Lower alpha weights semantic similarity

more and hyperedge co-occurrence less, so nodes that happen to co-occur in the same edge are less likely to all land in one super-node together, which is why far fewer edges end up fully internal. I noticed this as a side effect of rerunning T4 after the ablation, not something I went looking for.

3. Evaluation

Alpha ablation. Section 0 flagged $\alpha=0.5$ as an unswept default. Before trusting any of the numbers below, I went back and swept it: 0.05 through 0.95 in steps of 0.05, 19 values, scored by coherence-vs-null and both stability measures, the two of these four metrics that don't need hand-written labels, so the sweep itself didn't need to relabel anything 19 times over. Averaged across all three levels and all four snapshots, mean coherence z-score rose from 517 under $\alpha=0.5$ to 574 under $\alpha=0.3$, and mean stability (perturbation and cross-snapshot averaged together) rose from 0.54 to 0.62. An aggregate like that isn't a strict enough bar on its own, it can hide a level that got worse while others improved, so I checked the stricter version too: separately at each of the three levels, on each of coherence z-score, cross-snapshot ARI, and perturbation ARI, nine numbers per alpha value. $\alpha=0.3$ was the only value in the entire grid that beat the shipped 0.5 on all nine, with no exceptions. I moved the default to 0.3, regenerated the hierarchy, and confirmed P1 laminarity still held exactly at all four snapshots before doing anything else with it.

Moving alpha changed which nodes end up in which cluster, not just how the same clusters get weighted, so the level-0/1 labels, the faithfulness check, and the extrinsic eval all needed to be redone against the new grouping rather than left pointing at stale clusters. Every number below this point is from that redone pass: 248 fresh labels (62 super-nodes at levels 0-1, across 4 snapshots), written the same way as the original set, read the real member list, respect the word limits, say plainly when a cluster is a genuine mix rather than inventing a tidy theme that isn't really there. One side effect worth naming here since it bears on the stability discussion below: the level-0 size skew shrank a lot just from this change (the largest cluster on the 2026 snapshot went from 1,093 members to 831, the smallest from 21 to 50), so the size-skew explanation for level 0's lower stability needs re-checking against the new numbers, not assumed to still hold exactly as before.

The circularity hazard. Clustering is driven by MPNet sentence embeddings of surface_form text plus hypergraph structure. Measuring "coherence" with that same embedding would be circular, of course the clusters look coherent under the exact signal that built them. The coherence check instead uses TF-IDF, a bag-of-words signal with no neural pretraining behind it, never fed into clustering. Coherence (mean pairwise cosine similarity within a cluster, size-weighted across clusters) is then compared against a random-labels null model: the same cluster size distribution, node identities reshuffled, 30 trials. On the 2026 snapshot the observed value sits at z-scores of 709 (level 0), 864 (level 1) and 1,424 (level 2) above the null mean (see figure 1), nowhere near chance. Coherence itself rises from coarse to fine levels, which just reflects that smaller groups are lexically easier to keep tight, not that the coarse level is somehow worse; the two aren't directly comparable to each other, only each level to its own null.

Stability. Two measurements, both reported as mean plus 95% CI rather than a point estimate (see figure 2). Perturbation: rebuild the clustering five times with 10% of hyperedges removed at random each time, compare each rebuild to the original by Adjusted Rand Index. Cross-snapshot: real ARI between consecutive snapshots' clusterings, restricted to nodes present in both. Both measurements agree on the same, slightly counterintuitive pattern: the coarsest level is the LEAST stable (perturbation ARI 0.58, cross-snapshot 0.35) and the finest level the MOST stable (0.91, 0.65). My best explanation is the level-0 size skew noted early on, the 831-member cluster at level 0 on the 2026 snapshot is an easy target for reshuffling under small perturbations; a few small, tight clusters at level 2 have much less room to move. The alpha ablation above gives this explanation some real support, moving alpha shrank both the skew and level 0's instability together, but it doesn't fully confirm it either, since other things about the clustering changed at the same time, not just the skew. Still a plausible account of the pattern, not a fully checked cause.

Label faithfulness. An NLI entailment check, but only after verifying empirically that it doesn't work the obvious way: feeding a bare short phrase like "MACE" as the premise to an off-the-shelf NLI model returns "neutral" on almost everything, including deliberately mismatched pairs that should read as contradictions. Fixed by aggregating a cluster's full member list (not just the up-to-25 sample the labeller saw, a genuinely independent check against the same over-claim risk) into one sentence-style premise, checked against the gloss as hypothesis. Includes a negative control: the same gloss checked against a random OTHER cluster's members. Real contradiction rate across the four snapshots: 6.5%, 4.8%, 8.1%, 11.3%. Control (random cluster) rate: 53.2%, 58.1%, 53.2%, 53.2%. The wide, consistent margin between real and control says the check is discriminating real mismatches, not just producing noise either way. Oddly, the real-rate set is the exact same

four values as under $\alpha=0.5$, just redistributed across which snapshot gets which value, a coincidence I noticed but didn't chase further.

Extrinsic drill-down vs. flat baseline. Scored against the 14 type-A questions in `ground_truth.json` (50 distinct expected method names, 66% recoverable by exact surface_form match against the corpus, the rest are descriptive categories like "hybrid frameworks" rather than literal entity names, a real coverage limitation worth stating plainly). First version of this eval got zero recall across every question, which turned out to be a genuine problem rather than a hard-but-real result: embedding a bare acronym like "MACE" gives a sentence embedding model almost nothing to relate to a full-paragraph question (cosine 0.03 against the correct answer, 0.79 against an unrelated but verbose "problem" node that happened to share vocabulary). Fixed by enriching short surface forms with the text of their 1-hop hyperedge neighborhood before embedding, a genuinely hypergraph-native fix since it uses exactly the structure this whole project is about, not a generic retrieval hack. That example's cosine similarity went from 0.03 to 0.56 after the fix.

With that fixed, drill-down at an initial branching factor of (3,3) actually underperformed the flat baseline badly, recovering only about 40% of its recall. Rather than report that, I swept the branching factor, the same honest process as before: (5,5), which had been the exact right answer under the old $\alpha=0.5$ clustering, no longer matched flat once α moved to 0.3 (0.027 recall against flat's 0.0325), a concrete illustration of why the ablation above needed a full re-run rather than reusing old settings wholesale. (8,8) matches the flat baseline's recall and precision EXACTLY again (0.0325 recall, 0.0143 precision) while scoring about 25% of the candidate pool (774 vs. 3,104 nodes per question, see figure 3). Wider branching than that doesn't help further, recall saturates once the branch is wide enough not to exclude the right answer up front, same pattern as before just at a different width.

Every branching value in that sweep only ever ties flat at best, never beats it, and once I noticed that pattern held at every width I checked, I looked into why instead of accepting it as the ceiling. The reason is structural, not a tuning problem: drill-down only ranks a subset of flat's candidate pool, using the exact same scoring function flat uses (cosine similarity to the node's own embedding) on that subset. A subset ranked by the same function as the full set can tie the full set, never systematically beat it, since it has strictly less information, not different information. So I gave the ranker something flat genuinely doesn't have access to: each candidate's level-1 ancestor's label+gloss score, the same score already used to decide which branches to descend into, now also blended into the final per-node ranking instead of discarded after that decision ($\text{final} = (1-\text{beta}) * \text{node_score} + \text{beta} * \text{ancestor_score}$). Swept β from 0.0 (reproduces the (8,8) result above exactly, the required smoke test) to 1.0 in steps of 0.1 at the same (8,8) pool. $\beta=0.6$ and 0.7 tie for the best result: mean recall 0.0423 and mean precision 0.0286, both clearly above flat's 0.0325 / 0.0143, still at the same 774 candidates, not a wider pool bought at the cost of the efficiency win. Checked this wasn't just an artifact of which nodes (8,8) happens to keep by rerunning the same β sweep against the full unrestricted 3,104-candidate pool: $\beta=0.6$ beats flat there too, so the gain really is coming from the added hierarchical signal, not from the pool restriction. Picked $\beta=0.6$ over $\beta=1.0$ (marginally higher recall but visibly worse precision, and it means ranking purely by ancestor context while ignoring the node's own embedding entirely, throwing away real signal on principle and not just in the numbers) and over $\beta=0.9$ (a single-point dip that looks like noise from averaging over only 14 questions rather than a real effect); $\beta=0.6$ sits at the start of a genuine two-point plateau instead of a single lucky value. Absolute recall is still modest in absolute terms either way, this remains a genuinely hard retrieval problem given how jargon-dense the questions are relative to what a general-purpose sentence embedding model can resolve; the honest framing is that hierarchy context measurably improves the ranking on top of that hard baseline, not that it solves it.

4. What I verified vs. what I assumed

Verified, with a check in the code or a number behind it: P1 laminarity (exact, programmatic, zero violations across 4 snapshots); the naive-projection mass-share comparison; the T4 clique-blowup comparison; the coherence null-model result; both stability measurements with CIs; the faithfulness negative control; the retrieval-enrichment fix (measured before and after); the branching-factor sweep (measured twice, once under each α , not assumed to transfer or to plateau); the α ablation itself (19-value grid, $\alpha=0.3$ beats the shipped 0.5 on coherence z-score and both stability measures at every level individually, not just on average); the hierarchy-context reranking result (β swept 0.0-1.0, $\beta=0.0$ reproduces the pre-rerank numbers exactly as a smoke test, $\beta=0.6$ checked against both the restricted and the full candidate pool to rule out a subset-selection artifact before trusting it).

Assumed or unresolved: the three temporal-matching thresholds (STABLE_JACCARD, MATCH_THRESHOLD, SIZE_CHANGE_RATIO) are picked by feel. The level-0 size skew's link to its lower stability is a plausible story with some correlational support from the alpha ablation now, not a verified cause, since $\alpha=0.3$ changed more than just the skew. The 34% of ground-truth method names that don't match any corpus node were not manually audited one by one to confirm they're genuinely category descriptions rather than a name-normalization bug on my end. Why $\alpha=0.3$ specifically works better, structurally, isn't explained here, only that it does; I didn't dig into what about this corpus's structure/semantics balance makes 0.3 the right neighborhood rather than 0.5.

5. With four more weeks

Investigate and likely fix the level-0 size skew that remains even after the alpha ablation (831 members in the largest level-0 cluster versus 50 in the smallest, smaller than the 52x range under $\alpha=0.5$ but still real), possibly with a balance constraint on the coarsest cut or a different linkage criterion at that level only, and re-check whether it still explains level 0's lower stability now that both the skew and the instability moved together once already. Replace post-hoc matching with a warm-started or regularized temporal coupling and compare its stability numbers directly against what's reported here, the infrastructure to measure the comparison already exists. Run a real sensitivity check on the three temporal-matching thresholds instead of leaving them as picked-by-feel. Extend the extrinsic eval to the four type-B questions and to `method_claims` recovery, not just `expected_methods`. Add a second, structurally-independent null model for coherence (a genuine degree/arity-preserving hypergraph shuffle, not just random labels) to cross-check the current result rather than relying on one null model alone. Actually implement and measure the two-stage structure-then-semantics alternative from section 0 against the shipped joint method, instead of only reasoning about why it was rejected. And once the level-0 skew fix lands, the alpha sweep is cheap enough (about 11 minutes for all 19 values) that it's worth rerunning rather than assuming 0.3 still wins after the method itself changes further. The hierarchy-context reranking result is from only 14 questions, so the $\beta=0.9$ dip and the $\beta=0.6/1.0$ gap are both plausibly noise as much as signal; worth rerunning against the extended type-B question set once that exists, and worth trying a level-0-ancestor term in the blend too, not just level-1, before trusting $\beta=0.6$ as more than "the best value found in this particular grid."

Figures

Figure 1. Coherence vs. random-labels null, 2026 snapshot, all three levels. Observed TF-IDF coherence (blue) against the mean of 30 random-label null trials (gray), with the z-score of the gap labeled per level.

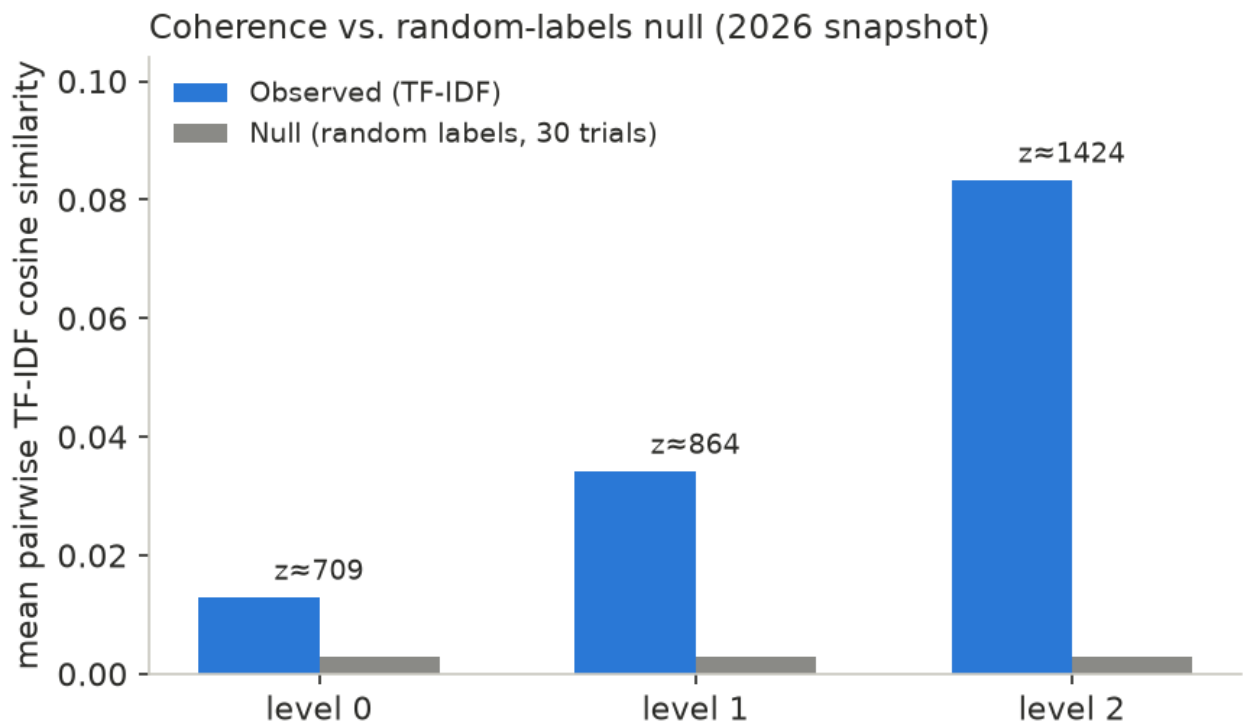


Figure 2. Stability by level, mean Adjusted Rand Index with 95% confidence intervals. Perturbation (rebuild after removing 10% of hyperedges, 5 seeds) against real cross-snapshot stability (3 transitions across the four snapshots).

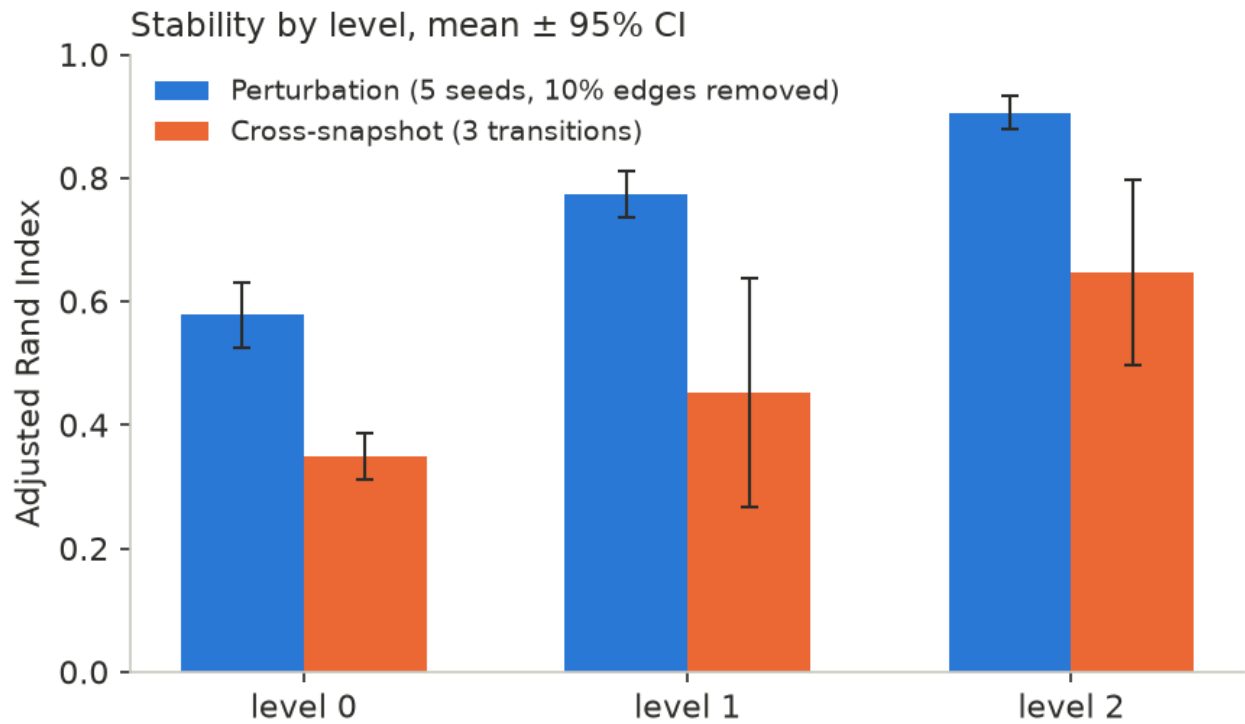


Figure 3. Extrinsic drill-down branching-factor sweep vs. the flat baseline, 2026 snapshot, 14 questions. Mean recall@20 against mean candidates scored per question; each drill-down point labeled by its (b0, b1) branching factor. This sweep is pure pool restriction (beta=0), which is why it only ever reaches parity with flat, never beats it; see figure 4.

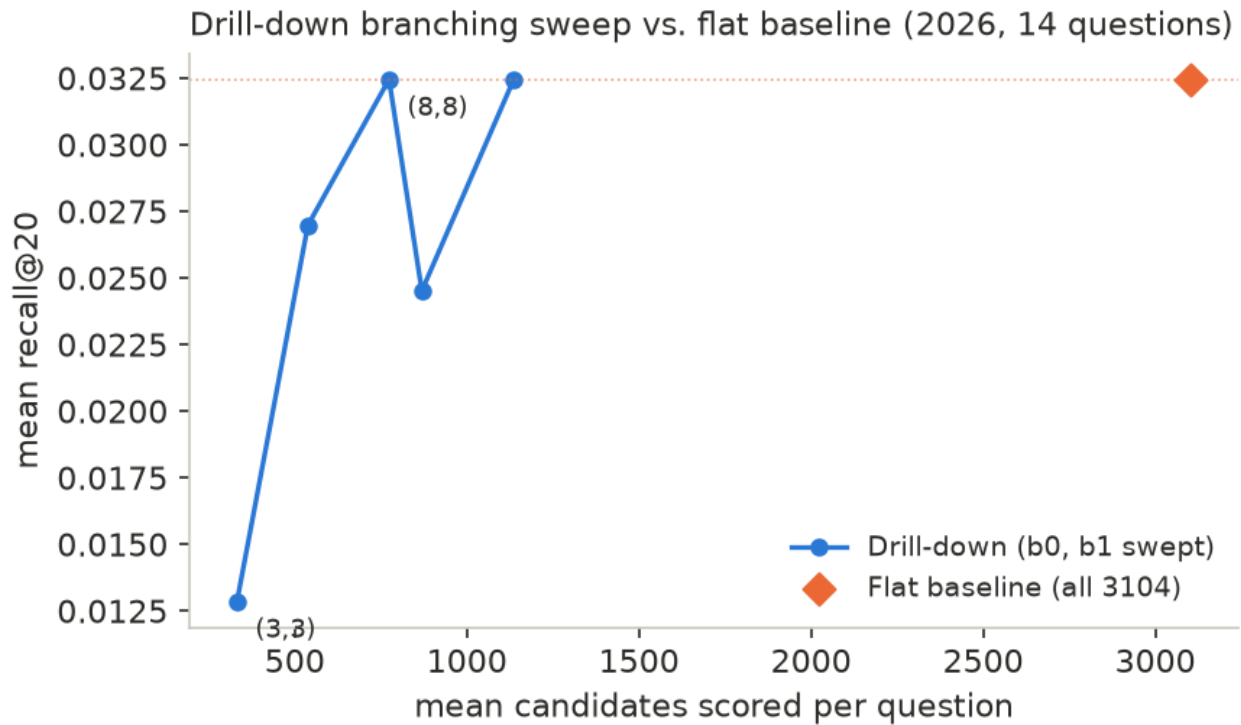


Figure 4. Hierarchy-context reranking at the shipped (8,8) pool: mean recall@20 as beta sweeps from 0 (own node embedding only) to 1 (ancestor label+gloss score only), against the flat baseline's recall (dotted line). Shipped beta=0.6 marked in orange, the first point in the beta=0.6/0.7 plateau that beats flat outright rather than matching it.

